

UK PATENT APPLICATION

SYSTEM AND METHOD FOR AUTONOMOUS CROSS-DOMAIN CAUSAL DISCOVERY AND BUSINESS GENERATION USING ENSEMBLE PREDICTION AND TEST-DRIVEN DEVELOPMENT

CLAIMS

Claim 1. A computer-implemented system for autonomous cross-domain causal discovery and business generation, the system comprising:

a processor;

a memory storing instructions that, when executed by the processor, cause the system to perform operations comprising:

a multi-source data ingestion module configured to collect time-series data from a plurality of heterogeneous external data sources spanning at least two data source categories, wherein a first category comprises behavioural signal sources and a second category comprises macroeconomic outcome sources, and to store the collected data as standardised time-series records each comprising a variable identifier, a data source tag, a timestamp, and a numerical value;

a correlation analysis engine configured to compute pairwise statistical association measures across all active variables by calculating, for each variable pair (i, j) where $i < j$, a plurality of distinct correlation coefficients with associated p-values, and to identify pairs as statistically significant where p is below a predetermined significance threshold;

a bidirectional Granger causality testing module configured to test, for each significant cross-domain variable pair, both a forward null hypothesis that variable X does not Granger-cause variable Y and a reverse null hypothesis that variable Y does not Granger-cause variable X, using frequency-adaptive maximum lag selection wherein the maximum number of lags is determined by data frequency, and to assign a causal direction category selected from the group consisting of: x_to_y, y_to_x, bidirectional, and none;

a multi-model ensemble prediction engine configured to generate a combined prediction for each qualifying signal-target pair by computing a weighted combination of a plurality of statistical sub-model predictors, wherein the ensemble combination algorithm computes an up_score equal to the sum of weight multiplied by confidence for all sub-models predicting an upward direction and a down_score equal to the sum of weight multiplied by confidence for all sub-models predicting a downward direction, and determines a final direction as upward if up_score is greater than or equal to down_score and downward otherwise, and a final confidence as the minimum of the absolute difference between up_score and down_score divided by total weight and

one;

a cross-layer enforcement module configured to constrain the ensemble prediction engine to generate predictions exclusively for variable pairs wherein exactly one variable originates from the first data source category and one variable originates from the second data source category, the constraint being applied at the database query level as a filter condition during pair selection;

an exploitation recommendation scoring module configured to rank discovered opportunities by computing an opportunity score using a weighted formula comprising a plurality of scoring components;

an autonomous test-driven development code generation pipeline configured to convert exploitation recommendations into deployed web applications through a plurality of sequential phases comprising at least specification generation, failing test creation, implementation generation, and output validation, wherein the implementation generation phase accepts code achieving a test pass rate of at least a predetermined minimum threshold and supports a configurable maximum number of retry attempts;

a deployment automation module configured to create a code repository on a remote hosting service and deploy the generated application to a cloud platform without human intervention; and

an outcome measurement module configured to validate predictions against realised values by comparing predicted direction and magnitude against actual target variable movement, and to recalibrate ensemble sub-model weights based on accumulated validated prediction accuracy.

Claim 2. The system of claim 1 wherein the plurality of heterogeneous external data sources comprises at least six sources including: Wikipedia pageview data providing behavioural demand signals, Reddit activity data providing social engagement signals, Federal Reserve Economic Data providing macroeconomic indicators, stock market price and volume data providing financial outcome variables, GDELT event data providing geopolitical signals, and ArXiv publication data providing academic output variables.

Claim 3. The system of claim 1 wherein the plurality of distinct correlation coefficients comprises at least Pearson product-moment, Spearman rank-order, and Kendall tau-b correlation coefficients, and wherein the predetermined significance threshold is $p < 0.05$, and the correlation analysis engine is configured to perform temporal alignment of variable pairs via outer-join alignment on timestamps with linear interpolation, and to require a minimum of twenty aligned data points before computing correlation coefficients.

Claim 4. The system of claim 1 wherein the bidirectional Granger causality testing module applies frequency-adaptive maximum lag selection comprising: two hundred and fifty-two lags for daily frequency data, fifty-two lags for weekly frequency data, twelve lags for monthly frequency data, four lags for quarterly frequency data, and two lags for yearly frequency data, with a minimum buffer of five to ten periods to ensure sufficient degrees of freedom, and requires a minimum of thirty aligned data points.

Claim 5. The system of claim 1 wherein the plurality of statistical sub-model predictors comprises a Granger causality predictor, an ordinary least squares regression predictor, and an autoregressive integrated moving average predictor, and wherein default sub-model weights are zero point four zero for the Granger causality predictor, zero point three five for the ordinary least squares regression predictor, and zero point two five for the autoregressive integrated moving average predictor.

Claim 6. The system of claim 5 wherein the ensemble prediction engine further comprises an automatic weight calibration mechanism that, when at least ten validated predictions exist for a given signal-target pair, computes per-model accuracy as the ratio of correct predictions to total predictions for each sub-model, and normalises the accuracy values to derive calibrated weights such that the calibrated weight for each sub-model is proportional to its accuracy divided by the sum of all sub-model accuracies.

Claim 7. The system of claim 1 wherein the cross-layer enforcement module defines Layer 1 sources as comprising variables with source identifiers "wikipedia" and "reddit" representing behavioural signals, and Layer 2 as comprising all remaining variables representing macroeconomic outcomes, and wherein the enforcement constraint is expressed as a SQL-level filter requiring that exactly one variable in each pair belongs to Layer 1 and exactly one belongs to Layer 2.

Claim 8. The system of claim 1 wherein the plurality of scoring components comprises demand, growth, timing, evidence, and competition, and the exploitation recommendation scoring module computes the opportunity score using the formula: $\text{opportunity_score} = (\text{demand_score} \times 30) + (\text{growth_score} \times 25) + (\text{timing_score} \times 20) + (\text{evidence_score} \times 15) + (\text{competition_score} \times 10)$, wherein each component score is normalised to the range zero to one and the component weights sum to one hundred.

Claim 9. The system of claim 1 wherein the plurality of sequential phases further comprises a code refinement phase, and wherein the predetermined minimum threshold is eighty percent, and the configurable maximum number of retry attempts is five, and wherein the autonomous test-driven development code generation pipeline comprises:

- a specification generation phase that analyses the exploitation recommendation text and generates a structured YAML specification comprising acceptance criteria, integration test scenarios, and end-to-end test scenarios, wherein the number of acceptance criteria is determined by a complexity tier selected from the group consisting of: LOW comprising four acceptance criteria, MEDIUM comprising seven acceptance criteria, and HIGH comprising twelve acceptance criteria;

- a failing test creation phase that generates test files from the YAML specification and creates a stub module that raises `NotImplementedError` for all imported symbols;

- an implementation generation phase that extracts required symbols from test imports, submits test code and symbol requirements to an AI language model, computes a pass rate as tests passed divided by the sum of tests passed, tests failed, and tests errored, and accepts the implementation if the pass rate is at least eighty percent;

a code refinement phase that submits the passing implementation to the AI language model for quality improvement and retains the pre-refinement version if the refined version introduces test failures; and

an output validation phase that performs abstract syntax tree parsing, compilation checking, and a final test execution.

Claim 10. The system of claim 9 wherein the implementation generation phase carries forward failure output from each unsuccessful attempt as context for subsequent retry attempts, enabling the AI language model to generate corrective code informed by previous failure modes.

Claim 11. A computer-implemented method for autonomous cross-domain causal discovery and business generation, the method comprising:

collecting time-series data from at least six heterogeneous external data sources spanning behavioural signal sources and macroeconomic outcome sources;

computing pairwise statistical association measures across all active variables using at least Pearson, Spearman, and Kendall correlation coefficients;

testing bidirectional Granger causality for each significant cross-domain variable pair using frequency-adaptive lag selection to determine temporal causal direction;

generating ensemble predictions by computing a weighted combination of Granger causality, ordinary least squares regression, and autoregressive integrated moving average sub-model predictions, wherein the combination algorithm computes directional scores as the sum of weight multiplied by confidence for each direction and selects the direction with the higher score;

enforcing cross-layer constraints at the database query level to ensure predictions flow exclusively from behavioural signal variables to macroeconomic outcome variables;

ranking discovered opportunities using a five-component weighted scoring formula;

converting high-scoring opportunities into deployed web applications via a five-phase test-driven development pipeline comprising specification generation, failing test creation, implementation generation with at least eighty percent pass rate acceptance, code refinement, and output validation;

deploying generated applications to cloud infrastructure without human intervention; and

validating predictions against realised values and recalibrating ensemble sub-model weights based on accumulated validated prediction accuracy.

Claim 12. The method of claim 11 further comprising engineering features from signal time-series by computing rolling mean, rolling standard deviation, and rolling coefficient of variation over three window sizes of seven, thirty, and ninety periods prior to generating ensemble predictions.

Claim 13. The method of claim 11 wherein the implementation generation step comprises up to five retry attempts, each attempt incorporating failure output from the preceding attempt as additional context for the AI language model.

Claim 14. The method of claim 11 further comprising classifying build failures across seven error categories comprising syntax errors, test failures, frontend errors, import errors, wiring errors, configuration errors, and runtime errors, and storing the error classification as structured data on the build record.

Claim 15. The method of claim 11 wherein deploying generated applications comprises creating a repository on a remote hosting service using a Git Tree API to push generated files as a single commit, and deploying to a cloud platform using a GraphQL API comprising project creation, service creation, and service instance deployment mutations.

Claim 16. The method of claim 11 further comprising detecting stale builds that have remained in any active processing state for longer than six hundred seconds and automatically marking the stale builds as failed.

Claim 17. The method of claim 11 further comprising measuring actual outcomes for exploitation recommendations by computing actual percentage change as the difference between target variable value at the end of the opportunity window and the baseline value at the start of the opportunity window divided by the absolute value of the baseline value multiplied by one hundred.

Claim 18. A non-transitory computer-readable storage medium storing instructions that, when executed by a processor, cause the processor to perform operations comprising:

collecting time-series data from at least six heterogeneous external data sources;

computing pairwise correlation coefficients and bidirectional Granger causality tests across all variable pairs with cross-domain enforcement;

generating ensemble predictions using a weighted combination of at least three statistical sub-models with automatic weight calibration from historical accuracy;

converting statistically significant causal signals into deployed web applications via an autonomous test-driven development pipeline; and

validating predictions against realised outcomes and recalibrating model weights accordingly.

Claim 19. The computer-readable storage medium of claim 18 wherein the automatic weight calibration comprises computing per-model accuracy from at least ten validated predictions and normalising accuracy values to derive calibrated weights proportional to each sub-model's empirical accuracy.

Claim 20. The computer-readable storage medium of claim 18 wherein the cross-domain enforcement constrains predictions to flow from variables originating from behavioural signal sources to variables originating from macroeconomic outcome sources by applying a database-level filter requiring exactly one variable in each pair to originate from a first source category and exactly one from a second source category.

Claim 21. The computer-readable storage medium of claim 18 wherein the ensemble predictions are generated by computing:

up_score = Sum(weight_i x confidence_i) for all sub-models predicting upward direction;

down_score = Sum(weight_i x confidence_i) for all sub-models predicting downward direction;

final_direction = upward if up_score >= down_score, otherwise downward; and

final_confidence = min(|up_score – down_score| / Sum(weight_i), 1.0).

Claim 22. The computer-readable storage medium of claim 18 wherein the autonomous test-driven development pipeline classifies requirement complexity into tiers comprising LOW with four acceptance criteria, MEDIUM with seven acceptance criteria, and HIGH with twelve acceptance criteria.

Claim 23. The computer-readable storage medium of claim 18 wherein the operations further comprise executing the data collection, correlation analysis, Granger causality testing, prediction generation, and outcome validation on an automated daily schedule without human initiation.

Claim 24. The system of claim 1 wherein the three-model ensemble prediction engine further comprises:

a feature engineering module that computes, for each signal time-series, rolling mean, rolling standard deviation, and rolling coefficient of variation over window sizes of seven, thirty, and ninety periods;

wherein the ordinary least squares regression predictor requires a minimum of six months of aligned training data and derives confidence from the R-squared coefficient of determination; and

wherein the autoregressive integrated moving average predictor uses order (1, 1, 1) and requires a minimum of twenty-four data points.

Claim 25. The system of claim 1 wherein the outcome measurement module further comprises a walk-forward backtesting sub-module that, for each signal-target pair with sufficient data, performs expanding-window ordinary least squares regression starting from a minimum training window of six data points, iterating through all subsequent data points, recording direction correctness at each split, and storing backtest predictions with a model version identifier of "backtest_walkforward."

Claim 26. The system of claim 1 wherein the exploitation recommendation scoring module classifies each recommendation into an action type selected from the group consisting of: BUY, SELL, BUILD, and MONITOR, wherein the BUILD action type triggers the autonomous test-driven development code generation pipeline.

Claim 27. The system of claim 1 wherein the data ingestion module executes on a scheduled daily cycle comprising: ingestion of Wikipedia pageview data at 01:00 UTC, ingestion of Reddit activity data at 01:30 UTC, ingestion of geopolitical event data at 02:00 UTC, prediction validation at 02:30 UTC, and ensemble prediction generation at 05:00 UTC, with weekly operations comprising baseline snapshot at 03:00 UTC on Sunday and walk-forward backtesting at 04:00 UTC on Monday.

Claim 28. A computer-implemented build iteration system for self-correcting autonomous code generation in an autonomous cross-domain causal discovery and business generation pipeline, the system comprising:

- a processor;

- a memory storing instructions that, when executed by the processor, cause the system to perform operations comprising:

- a failure diagnosis module configured to classify errors in a failed software build across at least seven error categories comprising syntax errors, test failures, frontend errors, import errors, wiring errors, configuration errors, and runtime errors, and to store the error classification as structured data;

- an iteration context generator configured to collect the parent build's error patterns, generated file contents, and current ensemble predictions for related statistical signals;

- a child build spawner configured to create a new build record linked to the parent build via a `parent_build_id` reference with an incremented `iteration_number` and an `iterate_reason` derived from the failure diagnosis; and

- a concurrency guard configured to prevent spawning a new iteration if an active build in the same iteration chain has a status of QUEUED or GENERATING.

Claim 29. The build iteration system of claim 28 wherein the child build spawner enriches the build specification with the parent build's error breakdown and failing test output, enabling the code generation pipeline to produce corrective implementations informed by previous failure modes.

Claim 30. The build iteration system of claim 28 further comprising an iteration chain query module configured to return all builds sharing a common root ancestor ordered by iteration number, enabling tracking of the self-correction trajectory.

Claim 31. The build iteration system of claim 28 wherein the `iterate_reason` comprises a diagnostic summary identifying the predominant error category from the parent build's error breakdown and specific error messages from failing test output.

Claim 32. The system of claim 1 wherein the correlation analysis engine stores, for each variable pair, a unified statistical profile comprising correlation value, absolute correlation for ranking, p-value, sample size, source identifiers for both variables, Granger p-values for both test directions, optimal lag count, and causal direction category.

Claim 33. The system of claim 1 wherein the deployment automation module further comprises a stale-build detection mechanism configured to monitor all builds in an active processing state and to mark as failed any build that has remained in state QUEUED, GENERATING, UPLOADING, or DEPLOYING for longer than a configurable timeout period, preventing indefinite resource consumption.

Claim 34. The method of claim 11 wherein the bidirectional Granger causality testing assigns causal direction categories as follows: x_to_y when the forward test is significant and the reverse test is not significant, y_to_x when the reverse test is significant and the forward test is not significant, bidirectional when both tests are significant, and none when neither test is significant, at a significance threshold of $p < 0.05$.

Claim 35. A computer-implemented commercial intelligence system for optimising autonomous code generation in the system of claim 1, the system further comprising:

- a product deployment tracking module configured to automatically create a deployment record upon successful completion of each build, the deployment record comprising a build identifier, a recommendation identifier, a product name, an application identifier, a technology stack snapshot, a pricing model, a market category, a target demographic, a deployment URL, and a deployment status;

- a revenue aggregation module configured to receive payment processor webhook events comprising checkout completions, subscription updates, subscription cancellations, and invoice payments, and to aggregate the events into periodic metrics records comprising monthly recurring revenue in cents and net subscriber count per deployment;

- an engagement aggregation module configured to periodically query an analytics service for page views, unique visitors, and average session duration per deployed product hostname, and to store the results as periodic metrics records per deployment;

- a commercial ranking engine configured to compute a composite commercial score for each active deployment on a zero-to-one-hundred scale by calculating a weighted sum of four normalised dimensions comprising revenue at forty percent weight, engagement at thirty percent weight, conversion rate at twenty percent weight, and retention at ten percent weight;

- a winning configuration analyser configured to group deployments by technology stack, pricing model, market category, and target demographic, and to compute per-group average commercial scores identifying which configurations correlate with commercial success; and

a specification enrichment module configured to inject the commercial intelligence context into the artificial intelligence language model prompt during specification generation, the context comprising top-performing deployments, best-performing configurations, and market-specific insights.

Claim 36. The commercial intelligence system of claim 35 wherein the revenue aggregation module further comprises an idempotency guard that checks for existing aggregation records before creating new records, preventing duplicate aggregation from repeated webhook events.

Claim 37. The commercial intelligence system of claim 35 wherein the product deployment tracking module creates the deployment record within the same database transaction as the build completion, ensuring no successful build exists without a corresponding deployment entry.

Claim 38. The commercial intelligence system of claim 35 further comprising a configuration confidence scoring module configured to compute, for a proposed build configuration, a confidence score on a zero-to-one-hundred scale by:

inspecting each dimension of the proposed configuration against the winning configuration database;

for each dimension where the proposed value matches a historically successful configuration, deriving a per-dimension confidence from the historical average commercial score boosted by an evidence multiplier of five percentage points per prior deployment capped at twenty percentage points;

for each dimension with no historical precedent, assigning a baseline confidence of thirty;

averaging the per-dimension confidences to produce an overall confidence score; and

generating a plain-English recommendation selected from the group consisting of: "High confidence" when overall confidence is at least seventy-five, "Moderate confidence" when overall confidence is between fifty and seventy-four, and "Low confidence" when overall confidence is below fifty.

Claim 39. The commercial intelligence system of claim 35 further comprising a demographic pattern mapping module configured to group ranked deployments by target demographic segment and to compute, for each segment, the deployment count, average commercial score, most common technology stack, most common pricing model, and most common market category.

Claim 40. The commercial intelligence system of claim 35 wherein the specification enrichment module injects into the AI language model prompt a context block comprising: the top five commercially successful deployments with their composite scores and configuration details, the three highest-scoring technology stacks and pricing models, market-specific insights for the target market category if historical data exists, demographic-specific insights for the target demographic if historical data exists, and the

configuration confidence score with plain-English recommendation.

Claim 41. The commercial intelligence system of claim 35 wherein the commercial ranking, winning configuration analysis, revenue aggregation, and engagement aggregation are executed on a scheduled daily cycle, and wherein the specification enrichment is performed synchronously during each specification generation request, ensuring the AI language model receives the most recent commercial intelligence data available at the time of build initiation.